

Task -9

By: Arkusleo

Date: 2025-10-30

Target: Local Vuln-Bank at <http://localhost:5000>

Intro

I deployed the Vuln-Bank lab locally and performed targeted manual testing. I found multiple high-impact vulnerabilities that are trivially exploitable in this intentionally vulnerable app.

The top findings:

- 1) Admin privilege escalation (credentials / token weaknesses)
- 2) Business logic flaw that accepts **negative transfer amounts**
- 3) Stored XSS input accepted into user list. Each finding includes impact, and remediation.

What I found

1) Admin access/weak auth

- What happened: I could access the Admin panel. Source code shows a weak hardcoded JWT secret and tokens without expiry.
- Why it matters: With admin access you can reset passwords, move money, create admin accounts, or read PII. Full compromise.

2) Negative transfer accepted (amount = -1000) -> Critical

- What happened: Submitting a transfer with a negative amount was accepted and changed balances.
- Why it matters: You can increase your balance (or make others negative) theft possible.
- Intercept transfer request, set "amount":"-1000", resend. Refresh balances they changed.

3) Stored XSS in username field - High

- What happened: A username containing `<script>alert(1)</script>` was stored and appears in the admin user list.
- Why it matters: If that field is ever rendered unsafely (admin view, notifications), the script can run and steal admin session cookies.

Impact

- Admin takeover → full app control, data theft.
- Negative transfers → financial theft / accounting destruction.
- Stored XSS → session hijack, pivot to admin features.

ID	Vulnerability	Severity	Impact
1	Admin privilege escalation (weak auth / tokens / default secrets)	High / Critical	Full control of application & funds; create admin accounts, password resets, view/export user data
2	Business logic: Negative transfer accepted (amount = -1000)	Critical	Funds manipulation / theft — attacker can inflate own balance or drain others
3	Stored XSS in Username / User list	High	Persistent XSS capable of session theft / CSRF if rendered/executed in other contexts

Fixes

1.Lock down auth

- Remove hardcoded JWT_SECRET. Use env var / secret manager.
- Add exp to JWT and validate it.
- Require admin MFA and strict server-side role checks.

2.Stop XSS

- Escape all user data on output.
- Sanitize or restrict username characters.
- Add a Content Security Policy (CSP).

3.Extras

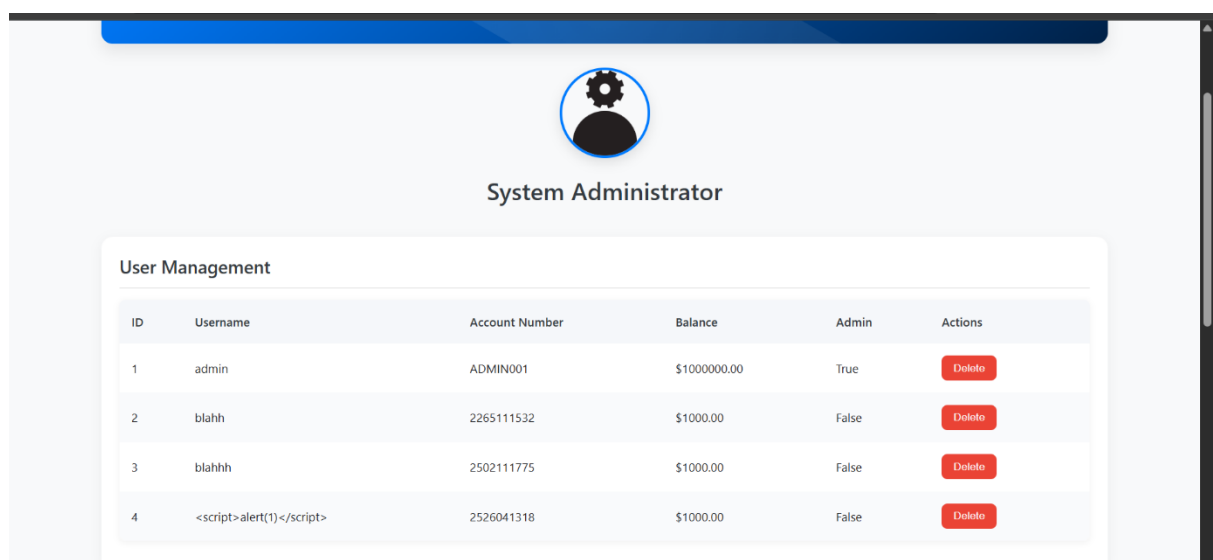
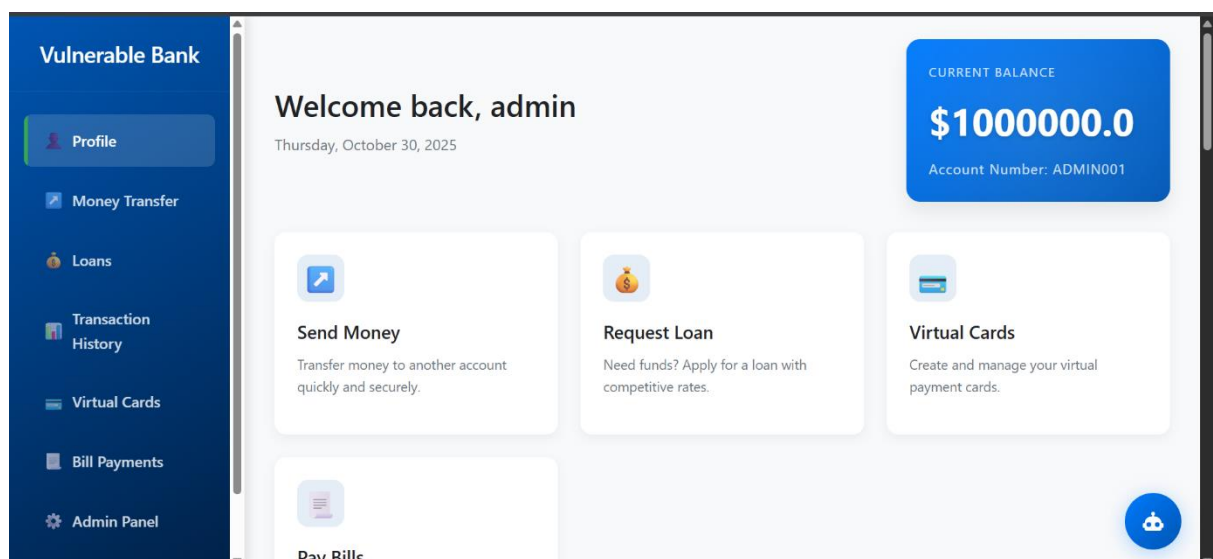
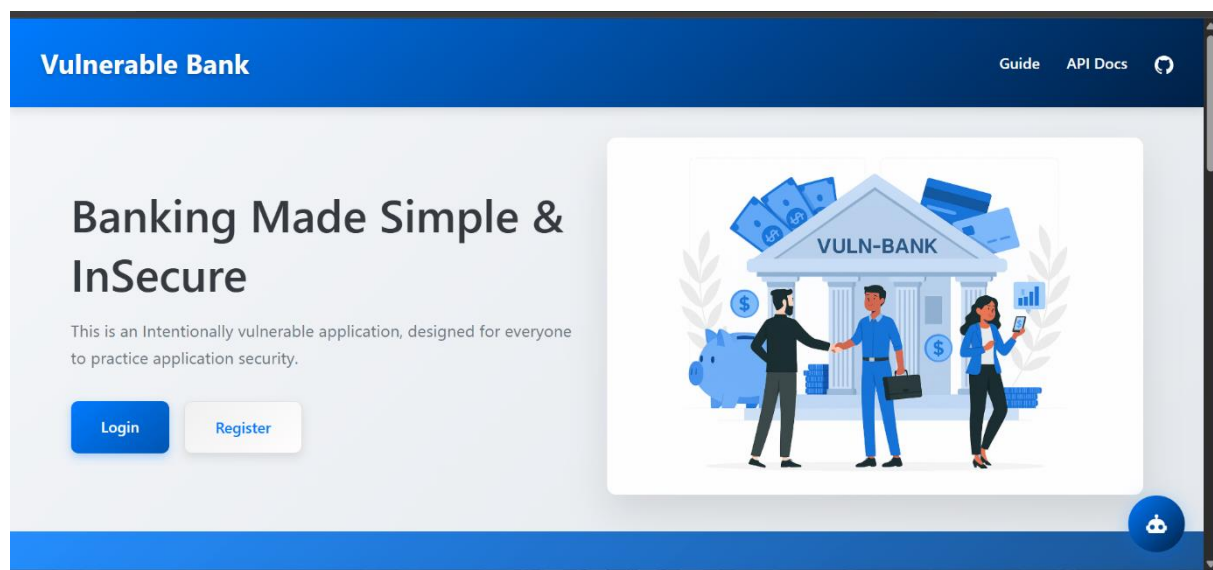
- Add logging + alerts for weird transfers.
- Add rate limiting on transfers.
- Add CI checks for secrets in repo.

Severity (quick)

- Negative transfer: **Critical**
- Admin auth weakness: **High** (rotate secrets + tokens)
- Stored XSS: **High**

Conclusion

uncovered multiple high-impact issues: admin takeover potential, ability to manipulate balances via negative transfers, and stored XSS. The business-logic issue in transfers is the most urgent fix server-side validation and add DB constraints immediately.



docker:desktop

PERSONAL

Q Search

Ctrl+K

?

1

A

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

vuln-bank

C:\Users\saksh\vuln-bank

View configurations

db

postgres:13

5432:5432

web

vuln-bank-we

5000:5000

Show all port

172.18.0.1

-

-

[30/Oct/2025 17:30:00]

"POST /login HTTP/1.1"

401

-

172.18.0.1

-

-

[30/Oct/2025 17:30:02]

"POST /login HTTP/1.1"

401

-

172.18.0.1

-

-

[30/Oct/2025 17:30:02]

"POST /login HTTP/1.1"

401

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"POST /login HTTP/1.1"

200

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /dashboard HTTP/1.1"

200

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /static/style.css

HTTP/1.1"

304

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /static/dashboard.js

HTTP/1.1"

304

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /static/dashboard.css

HTTP/1.1"

304

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /static/uploads/user.png

HTTP/1.1"

304

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /static/dashboard.js

HTTP/1.1"

304

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /api/virtual-cards

HTTP/1.1"

200

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /api/bill-categories

HTTP/1.1"

200

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /transactions/ADMIN001

HTTP/1.1"

200

-

172.18.0.1

-

-

[30/Oct/2025 17:30:05]

"GET /api/bill-payments/history

HTTP/1.1"

200

-

Engine running

RAM 0.94 GB CPU 0.25% Disk: 2.20 GB used (limit 1006.85 GB)

v4.49.0